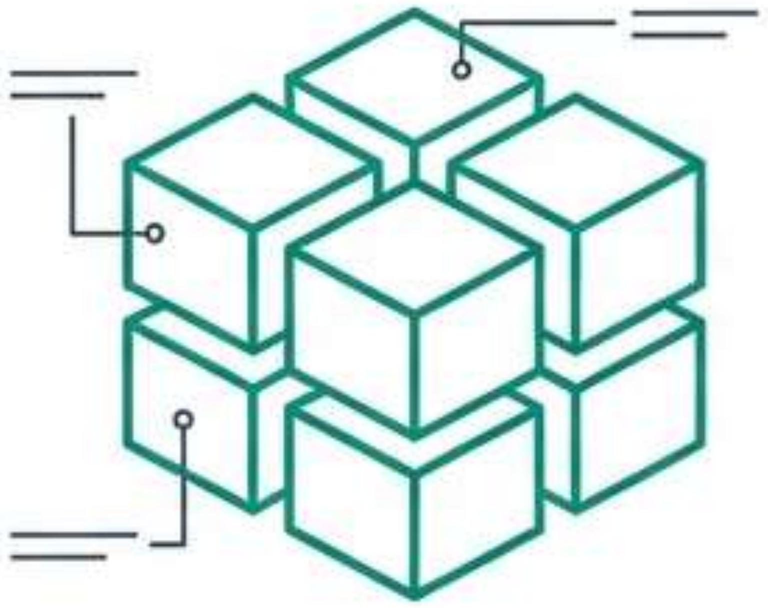
Mastering TestNG Annotations

Architecting Scalable Selenium Automation Frameworks

The Necessity of Structured Automation

Before

Unstructured Code



Annotated Framework

After

The Golden Circle of Framework Design

Why

To ensure maintainability, readability, and user-friendly debugging.

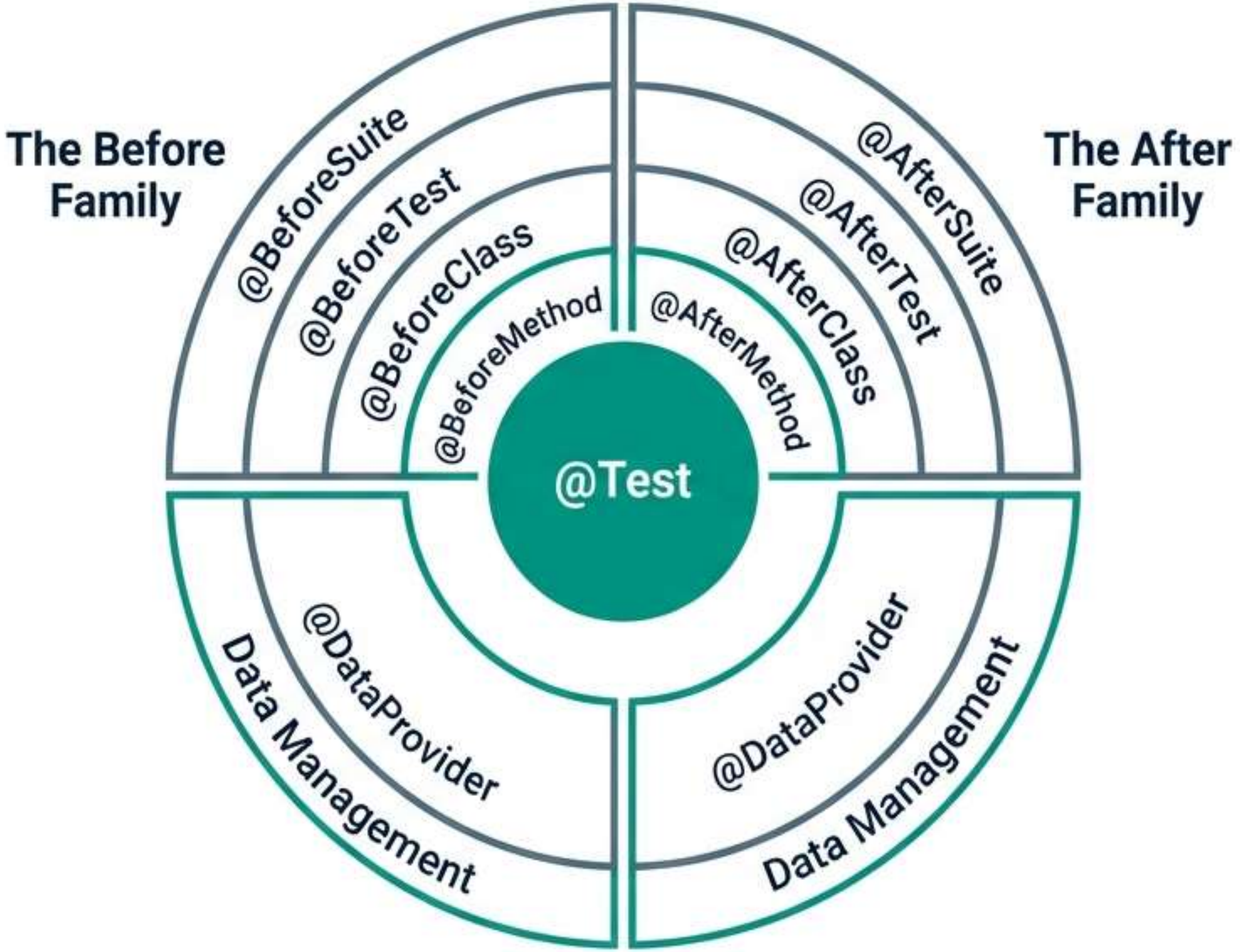
When

Applied during architecture to manage pre-conditions (setup) and post-conditions (teardown).

What

Metadata tags acting as instructions for the test runner.

The Annotation Ecosystem



Test Logic
The core method validating business logic.

Pre-Conditions
Setup drivers, DB connections, test data.

Post-Conditions
Browser closure, reporting, cleanup.

Data Management
Supplying complex data sets.

Understanding Execution Hierarchy

Testleaf
Always Ahead

Suite Level (@BeforeSuite) - Runs once per suite

Test Level (@BeforeTest) - Runs per <test> tag

Class Level (@BeforeClass) - Runs once per class

Method Level (@BeforeMethod) - Runs before *each* test

@Test

Method Level (@AfterMethod) - Runs after *each* test

Class Level (@AfterClass) - Runs once per class

Test Level (@AfterTest) - Runs per <test> tag

Suite Level (@AfterSuite) - Runs once per suite

Critical Distinction:
@BeforeClass executes once (efficient for DB).

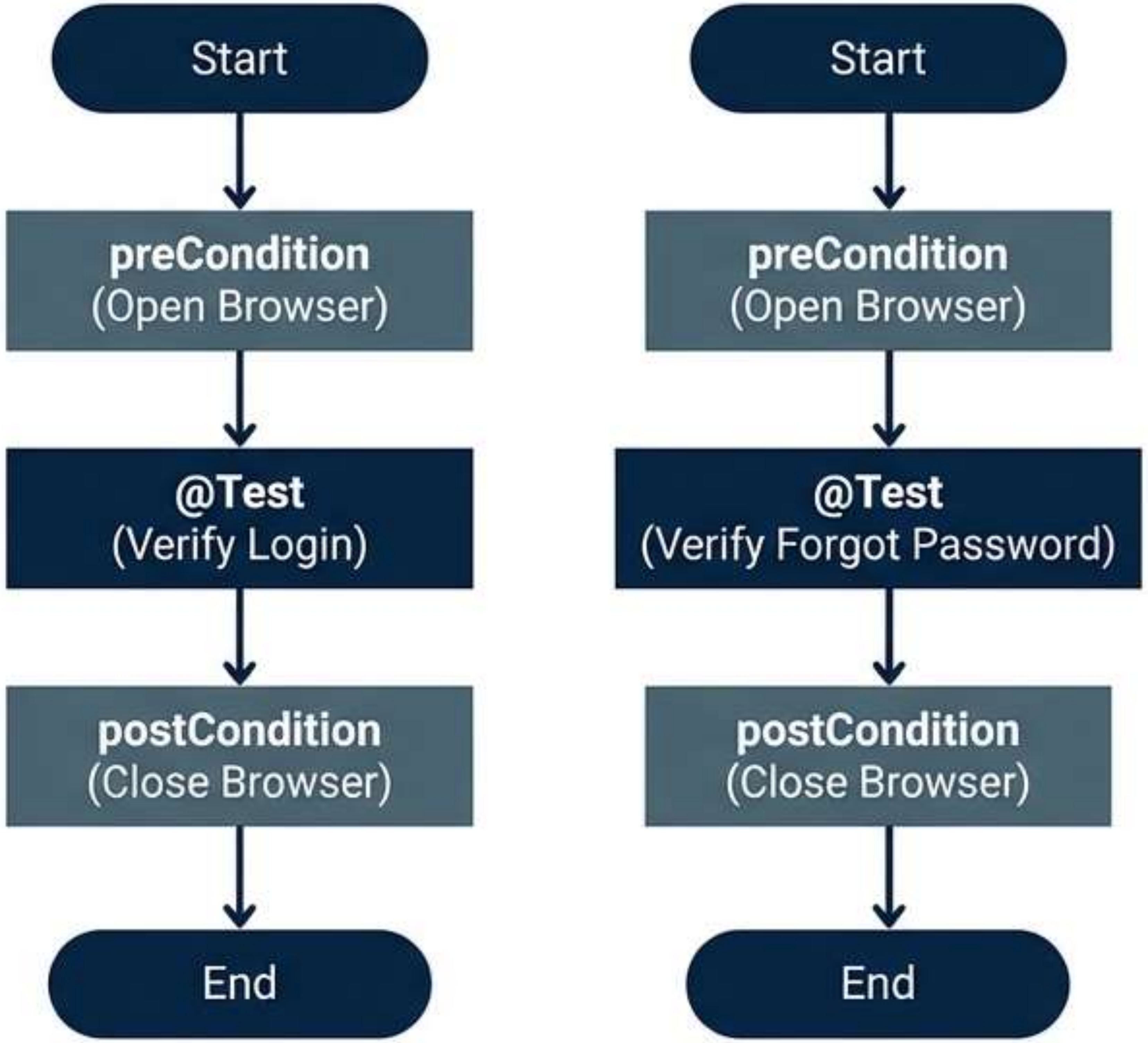
@BeforeMethod executes repeatedly (essential for clean browser sessions).

Practical Scenario: E-Commerce Login

```
public class BaseClass {
    public EdgeDriver driver;

    @BeforeMethod
    public void preCondition() {
        driver = new EdgeDriver();
        driver.get("http://...");
    }

    @AfterMethod
    public void postCondition() {
        driver.close();
    }
}
```



Testleaf
Always Ahead

Engineering Excellence in Annotations

Atomic Tests

Ensure every @Test is independent. Never rely on state from a previous test.

Appropriate Scoping

Use @BeforeClass for shared resources (DB), but @BeforeMethod for isolated browser sessions.

Descriptive Naming

Use names like `setUpBrowser()` instead of `init()` to aid debugging.

Robust Teardown

Always place cleanup code in @After methods to guarantee execution even after failures.

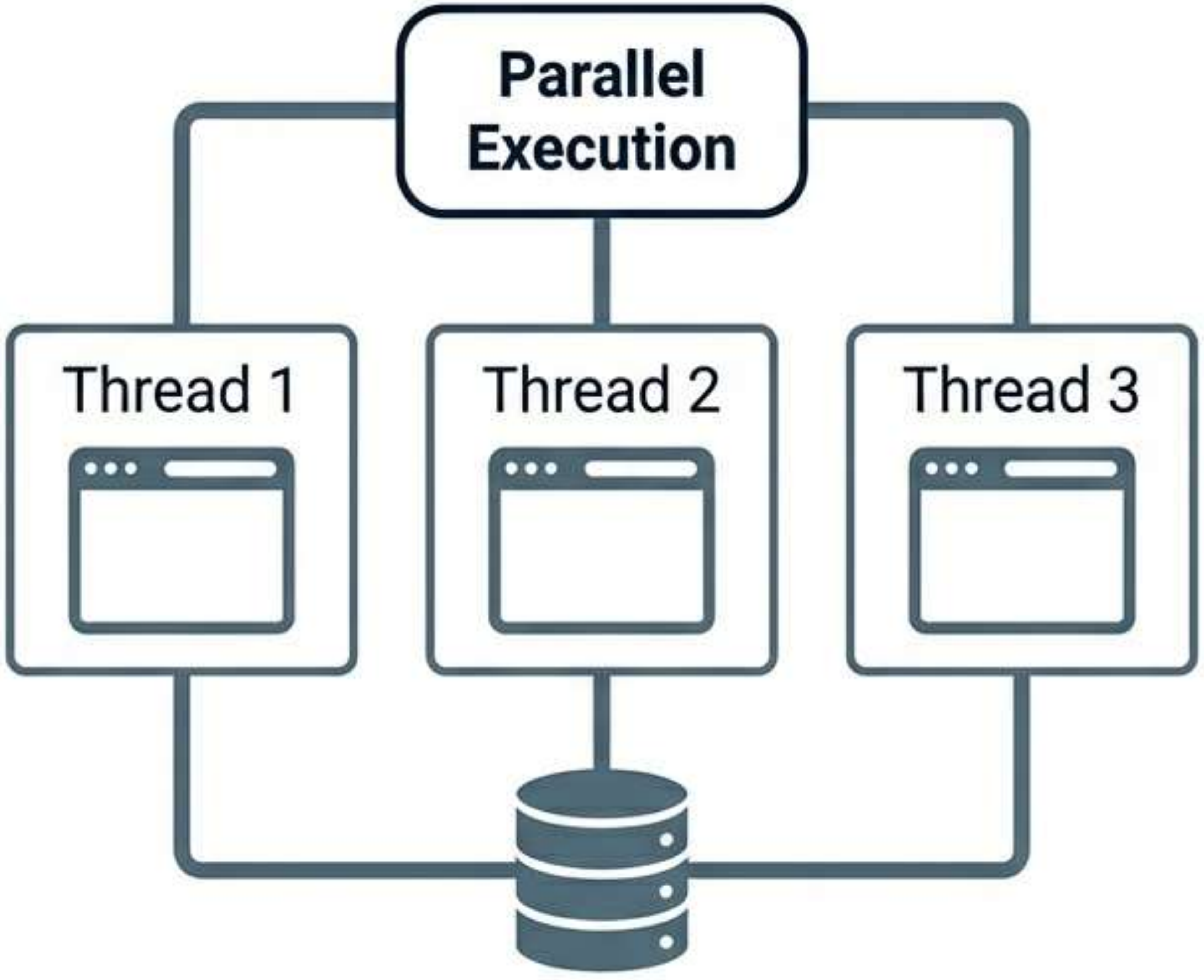
Testleaf
Always Ahead

Testleaf
Always Ahead

Challenge 1: Parallel Execution & Thread Safety

The Risk

**The Risk*: Static WebDrivers in a BaseClass cause session conflict (Test A controlling Test B's browser).



The Solution

- Avoid **static** WebDriver instances.
- Implement **ThreadLocal<WebDriver>** for isolation.
- Configure **testng.xml** with **parallel='methods'**.

Interview Readiness: Key Concepts

Execution Frequency

Q: Difference between @BeforeMethod and @BeforeClass?
A: @BeforeMethod runs N times (before every test). @BeforeClass runs 1 time per class.

Data Handling

Q: How to handle data-driven testing?
A: Use @DataProvider mapped to method parameters.

Scenario Check

Q: If a class has 5 tests, how many times does @BeforeMethod run?
A: Exactly 5 times.

Interviewers look for understanding of Lifecycle, Inheritance, and Thread Safety.

Summary of Core Competencies

- **Structure:** Annotations provide the architectural skeleton.
- **Efficiency:** The `BaseClass` pattern eliminates code duplication.
- **Control:** Hierarchy allows precise governance (Suite > Test > Class > Method).
- **Reliability:** Proper setup/teardown ensures clean environments and reduces flakiness.

Conclusion

Mastering TestNG annotations is about more than syntax—it is about designing a framework that is readable, maintainable, and scalable. By strictly separating test logic from configuration, you ensure your automation delivers reliable results.